The Open Source Audit
A Capstone Project Report

on

Apache HTTP Server

Submitted by:

Shashi Singh

Reg No: 24BAI10686

Course: Open Source Software

Slot: A24

Date of Submission:

Mar 30, 2026

1. Origin and Philosophy

1.1 The Problem Apache HTTP Server Was Created to Solve

In the early 90s, the web was just starting out. Most sites ran on a program named NCSA HTTPd - pioneering stuff at the time. Yet despite its popularity, it didn't do everything well. Progress on updates began to stall, until work stopped altogether. That left users in a tight spot. Without regular bug fixes, problems piled up slowly but surely. New tools? Forget about them - they weren't coming. Support barely existed when people needed help the most. As traffic grew online, the gap between what the software could do and what the world expected only widened.

All at once, web access began growing fast, many groups needed space online. Still, they ran into slow speeds, rigid setups, few customization options. The code existed yet wasn't updated often; so coders built personal tweaks to repair flaws, add features. These adjustments spread across different places without order, bringing chaos instead of reliability. Stability became hard when every version followed its own path.

Out of nowhere, a team guided by Brian Behlendorf started working together because fixes kept piling up. Their shared work turned those scattered updates into one stronger web server. That project became what people now call the Apache HTTP Server. Some say the word "Apache" came from how patch-filled it looked at first, stitched loosely like scrap fabric.

Choosing to release Apache as open source instead of locking it down came from a clear purpose. Web access ought to stay free, the team felt, not handed over to one corporation's grip. Opening up the code meant people everywhere could take it, tweak it, build on it. Because of that freedom, fixes and upgrades poured in fast, fueled by shared effort. Stability grew. Security tightened. Functions multiplied, outpacing what older systems offered.

Built out of necessity, Apache stepped in when solid web servers were scarce during rapid online expansion. Patchwork tweaks became something stronger - a shared effort shaping much of today's web backbone.

1.2 The License - What It Actually Says

The Apache HTTP Server is released under the **Apache License 2.0**, which is a permissive open-source license. This license allows users a high degree of freedom while still maintaining certain legal protections for the original developers.

The Four Freedoms of Free Software

The idea of free software is based on four essential freedoms:

- **Freedom 0:** The freedom to run the program for any purpose without restrictions.
- **Freedom 1:** The freedom to study how the program works and modify it according to individual needs. This requires access to the source code.
- **Freedom 2:** The freedom to redistribute copies of the software to others.
- **Freedom 3:** The freedom to distribute modified versions so that improvements can benefit the community.

Apart from offering basic functionality, this server grants full rights to examine how it works. Modification is allowed once someone obtains the code, meaning changes can be made without restriction. Sharing happens openly - both unchanged copies and altered forms circulate freely among users.

Commercial Use and Changes Allowed

Commercial use stands permitted under the Apache License. Should a business choose, it may adopt the Apache HTTP Server, alter its code, then distribute the result within a proprietary offering. Profit-driven deployment does not violate terms.

Still, specific requirements apply

- The original license must be included
- Recognition should go to those who first created it
- The modified version must clearly indicate changes

Freed from public disclosure rules, firms can tweak the source without sharing changes - this freedom draws many toward the license.

Difference Between Apache License and GPL

The Apache License differs sharply from the GNU General Public License, though both are frequently mentioned together. One builds on permissive principles, whereas the other enforces strict sharing rules. Their core ideas pull in separate directions; flexibility here, control there. Each reflects a distinct view of software freedom.

- The **Apache License** is permissive. It allows modifications and commercial use without forcing users to share their changes.
- The **GPL** is a copyleft license. It requires that any modified version of the software must also be distributed as open source under the same license.

Choosing a license for my project, I'd lean toward Apache. It allows greater freedom while supporting community work alongside business use. Flexibility stands out here, opening doors without demanding strict reciprocity. That balance makes it stand apart from others in practice.

Free as in Cost vs Free as in Freedom

In open-source discussions, "free" does not only mean free of cost. There are two meanings:

- **Free as in free beer:** The software is available at no cost
- **Free as in freedom:** Users have the freedom to use, modify, and distribute the software

Apache HTTP Server represents both meanings. It is free to download and use, but more importantly, it gives users the freedom to customize and improve it according to their needs.

1.3 The Ethics of Open Source

Open source is not just a technical model of software development; it represents a broader philosophy about how knowledge and technology should be shared. It raises important ethical questions about ownership, collaboration, and fairness in the digital world.

Should all software be open source?

One might argue that opening up all software code could foster greater collaboration. Yet others believe keeping some programs closed protects innovation. Sharing everything freely may encourage transparency. Still, restricting access at times can support sustainable development. Some see openness as a path to equality. Others warn it risks undermining creators' efforts. Making every program available sounds ideal on the surface. In practice, though, balance matters more than absolutes.

Open source fuels openness, skill growth, and fresh ideas. Because the code sits out in the open, coders get a chance to see behind the scenes, tweak what exists, then craft something different using prior work as foundation. Efforts aren't wasted reinventing tools already built; instead, advancement moves faster. With eyes everywhere, flaws or sneaky functions are more likely to be spotted, which builds stronger confidence over time.

Still, opening every kind of software isn't always practical. Firms spend large amounts of effort, funds, and energy building closed programs, counting on them to bring income. Without the chance to earn back those costs, fewer might take risks creating new tools in specific areas. Meanwhile, particular applications handle confidential information or pose safety issues, exposing them completely could create serious problems.

Looking at it one way, open source deserves support in most situations. Yet room remains for private software when circumstances or goals make it more suitable.

Is it ethical for companies to profit from open-source software?

This question sparks ongoing discussion across the open-source community. While some organizations gain financial benefits, they often rely heavily on tools such as Linux and Apache.

Yet this approach aligns with ethics, since open-source agreements like the Apache License clearly permit business applications. Still, firms frequently return value by resolving flaws, boosting efficiency, or supporting progress financially. Such actions foster a balanced exchange among participants.

Still, some worry firms might benefit from collaborative efforts while offering little in return. When businesses gain significant revenue from open-source tools yet fail to share updates or aid the developers, perceptions shift toward exploitation. So fairness hinges on how much users give back compared to what they take. Ethical practice emerges not just in access, but in reciprocity.

"Standing on the shoulders of giants"

What we call progress often begins by using knowledge left behind by those who came before. Open-source thrives when people take earlier efforts and shape them into something fresh. Earlier breakthroughs become tools, not endpoints. One person's solution opens paths someone else walks later. Ideas pass forward, shift slightly, grow stronger. This sharing turns isolated advances into shared momentum.

Because code is shared openly, creators can build directly on what others made. Starting from scratch becomes rare when proven pieces are freely available. Innovation moves faster since

effort shifts toward fresh challenges. Complex tools reach wider audiences without gatekeeping. Progress grows not by isolation, but through reuse and collaboration.

Still, a few worry that leaning hard on current open-source options can dull creativity. Rather than grasp core ideas fully, coders could lean on ready-made components. Even so, in most cases, shared code removes barriers by fueling new thinking more than holding it back.

2. Linux Footprint

Installation Method

Apache HTTP Server was installed using the apt package manager on Kali Linux. This method simplifies installation by automatically resolving dependencies and configuring the software.

```
sudo apt update
sudo apt install apache2
```

Key Directories

From the system analysis, Apache is distributed across standard Linux directories:

- **Binary location:** /usr/sbin/apache2, found using which command
- **Configuration directory:** /etc/apache2
- **Log directory:** /var/log/apache2

The /etc/apache2 directory contains important configuration files such as: apache2.conf, ports.conf, sites-available and sites-enabled etc.

The /var/log/apache2 directory contains log files including: access.log, error.log and other_vhosts_access.log

These logs help in monitoring server activity and debugging issues.

```
ubuntu@ubuntu:~$ which apache2
/usr/sbin/apache2
ubuntu@ubuntu:~$ ls /etc/apache2
apache2.conf    conf-enabled  magic           mods-enabled  sites-available
conf-available  envvars       mods-available  ports.conf    sites-enabled
ubuntu@ubuntu:~$ ls /var/log/apache2
access.log  error.log  other_vhosts_access.log
ubuntu@ubuntu:~$ ps aux | grep apache2
root        6310  0.0  0.0   6816  4928 ?        Ss   06:14   0:00 /usr/sbin/apache2 -k start
www-data    6311  0.0  0.0 754204  5724 ?        Sl   06:14   0:00 /usr/sbin/apache2 -k start
www-data    6314  0.0  0.0 754204  5728 ?        Sl   06:14   0:00 /usr/sbin/apache2 -k start
www-data    6513  0.0  0.0   3628  1628 ?        Ss   06:14   0:00 /usr/bin/htcacheclean -d 120 -p /var/cache/
apache2/mod_cache_disk -l 300M -n
ubuntu      6863  0.0  0.0  18352  2176 pts/0    S+   06:16   0:00 grep --color=auto apache2
ubuntu@ubuntu:~$ ps -ef | grep apache2
root        6310       1  0 06:14 ?        00:00:00 /usr/sbin/apache2 -k start
www-data    6311    6310  0 06:14 ?        00:00:00 /usr/sbin/apache2 -k start
www-data    6314    6310  0 06:14 ?        00:00:00 /usr/sbin/apache2 -k start
www-data    6513       1  0 06:14 ?        00:00:00 /usr/bin/htcacheclean -d 120 -p /var/cache/apache2/mod_cac
he_disk -l 300M -n
ubuntu      6870    5149  0 06:17 pts/0    00:00:00 grep --color=auto apache2
ubuntu@ubuntu:~$
```

User and Group

Apache HTTP Server follows a privilege separation model for better security. From the process analysis, it is observed that the main (parent) process runs under the root user, while multiple worker processes run under the www-data user and group.

- **Parent Process:** root
- **Worker Processes:** www-data

This design ensures that while the server can start with necessary system privileges, the actual handling of client requests is done by a less-privileged user. As a result, even if a vulnerability is exploited, the impact is limited and does not compromise the entire system.

Service Management

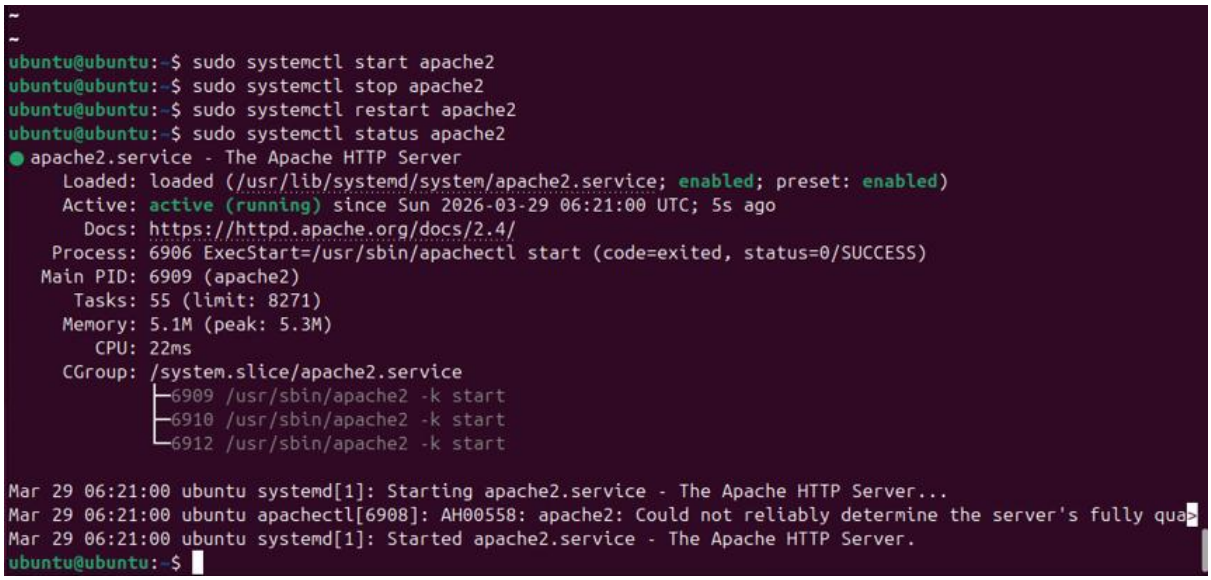
Apache runs as a system service managed through systemctl. The following commands were used:

```
sudo systemctl start apache2
sudo systemctl stop apache2
sudo systemctl restart apache2
sudo systemctl status apache2
```

The service status output showed:

- **Active state:** running
- **Main PID:** active process handling requests
- **Status message:** processing requests

This confirms that Apache is functioning correctly and handling web requests.



```
~
~
ubuntu@ubuntu:~$ sudo systemctl start apache2
ubuntu@ubuntu:~$ sudo systemctl stop apache2
ubuntu@ubuntu:~$ sudo systemctl restart apache2
ubuntu@ubuntu:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-29 06:21:00 UTC; 5s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 6906 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 6909 (apache2)
      Tasks: 55 (limit: 8271)
     Memory: 5.1M (peak: 5.3M)
        CPU: 22ms
     CGroup: /system.slice/apache2.service
             ├─6909 /usr/sbin/apache2 -k start
             ├─6910 /usr/sbin/apache2 -k start
             └─6912 /usr/sbin/apache2 -k start

Mar 29 06:21:00 ubuntu systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 29 06:21:00 ubuntu apachectl[6908]: AH00558: apache2: Could not reliably determine the server's fully qua>
Mar 29 06:21:00 ubuntu systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ubuntu:~$
```

Update Model

Apache updates are handled through the package manager:

```
sudo apt update
sudo apt upgrade apache2
```

Security patches and updates are released by the open-source community and distributed via repositories. This ensures that the system remains secure and up to date with minimal effort.

3. The FOSS Ecosystem Around Apache HTTP Server

Dependencies and Supporting Tools

Apart from core modules, Apache HTTP Server uses multiple open-source tools to operate effectively

- **APR (Apache Portable Runtime):** Provides a consistent interface to underlying OS features like file handling and networking.
- **APR-Util:** Extends APR with additional utilities such as database access and XML parsing.
- **OpenSSL:** Enables secure communication through HTTPS by providing SSL/TLS encryption.
- **PCRE (Perl Compatible Regular Expressions):** Used for pattern matching in configuration and URL rewriting.

These dependencies allow Apache to remain portable, secure, and highly customizable across different operating systems.

What Apache Has Enabled or Inspired

From its early days, Apache helped define how websites work today; because of it, countless software solutions emerged. Though often unseen, the server's influence stretches across online infrastructure. Where digital services exist, traces of this platform frequently appear. Its code became a foundation others built upon quietly yet effectively. As internet usage grew, so did reliance on what began as a modest project

- The system formed the core support structure for initial web hosting setups.
- A ripple effect followed, nudging projects like Nginx toward rethinking speed when systems strain. Heavy demand became a catalyst, shaping how these platforms handle pressure differently.
- Most websites built with tools such as WordPress use Apache to deliver pages. Yet some choose alternatives based on specific needs. Still, Apache remains a common choice across platforms. While performance matters, compatibility often drives this decision. Even newer setups sometimes stick with it despite other options existing.
- With modular extensions, it lets developers include extras such as authentication, caching, or scripting tools. While building outward, the system accepts these additions seamlessly through modules designed for flexibility.

Fueled by its adaptability, Apache has become central to many online platforms. Though built openly, it powers a vast share of internet infrastructure. Where customization matters, this

software often underpins digital operations. Because access remains unrestricted, developers rely on it widely. Its design allows adjustments without barriers, fitting diverse needs quietly.

Role in the LAMP Stack

Apache is a core component of the **LAMP stack**, which includes:

- **Linux** (Operating System)
- **Apache** (Web Server)
- **MySQL** (Database)
- **PHP** (Server-side scripting language)

In this stack, Web traffic flows through Apache first, since it serves as the main receiver of HTTP requests while sending back website data. This setup links user browsers to deeper layers such as databases or processing scripts. Its role sits at the core of how web apps are built and function.

Community and Governance

Apache HTTP Server is maintained by the **Apache Software Foundation (ASF)**, a non-profit organization that supports many open-source projects.

The Apache community is large and active, and it includes:

- Mailing lists for developers and users
- Online forums and documentation
- IRC channels and discussion platforms
- Conferences and events related to open source

The governance model is community-driven, meaning decisions are made collaboratively by contributors rather than controlled by a single company. This ensures transparency, continuous improvement, and long-term sustainability of the project.

4. Open Source vs Proprietary (Critical Analysis)

Dimension	Apache HTTP Server (Open Source)	Microsoft IIS (Proprietary)
Cost	Free to use, modify, and distribute	Requires Windows Server license (paid)
Security	Source code is open for public auditing; vulnerabilities can be identified and fixed by the community	Source code is closed; users must rely on Microsoft for security updates
Support & Reliability	Community support via forums, documentation, and contributors; reliable but may require self-troubleshooting	Official support from Microsoft with dedicated services and documentation
Freedom to Modify	Full freedom to modify source code and customize behavior	Limited customization; no access to source code

Community vs Control	Community-driven development under Apache Software Foundation	Fully controlled and developed by Microsoft
Overall Verdict	Best suited for flexibility, cost-effectiveness, and open environments	Better suited for enterprise Windows-based ecosystems with official support

Given the findings from this study, choosing Apache HTTP Server appears sensible when deploying systems where openness, low cost, and adaptability matter most. Because it is open source, teams can modify how it works to match exact requirements like something rigid platforms often lack. Updates keep coming, thanks to active contributors who fix issues and add features regularly. Being able to inspect every line of code helps verify safety, which builds confidence over time. Even though setup might take extra effort versus closed alternatives, having full command over behavior pays off across diverse uses. What stands out is not just performance but long-term fit within evolving infrastructures.

Down the line, I'd like to get involved with Apache HTTP Server. Working on something that matters could sharpen how I think about complex systems while making the tool better for others. Learning happens faster when ideas move between people - especially those who've been around the block. Projects shaped by communities reflect a deeper idea: progress thrives when effort isn't hoarded. What sticks after all is shared work, not just code.

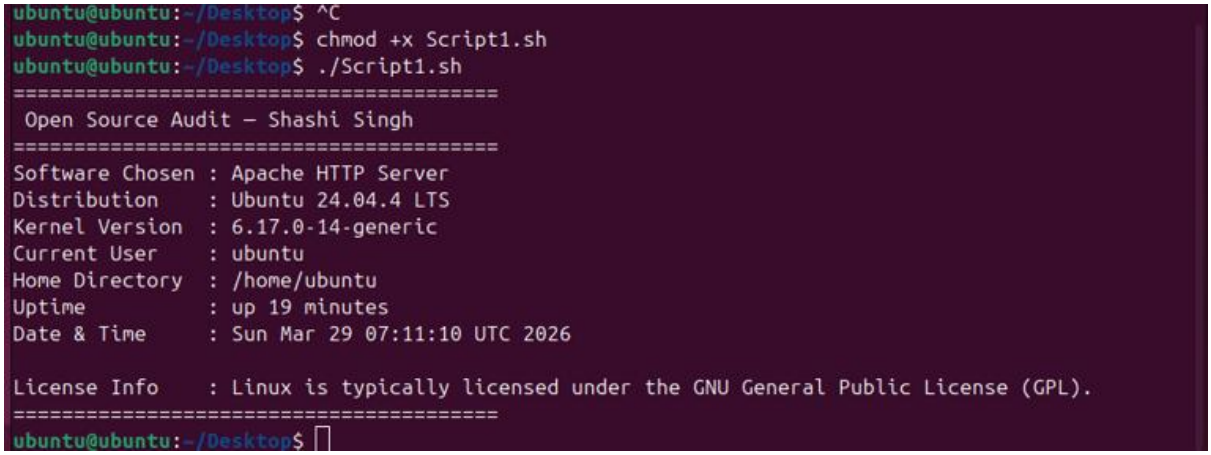
Shell Script Tasks

Script 1 — System Identity Report

Script Code:

```
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: Shashi Singh
# Course: Open Source Software
# Description: Displays important system information
# ============================================================

# ----------- User-defined Variables ------------------------
STUDENT_NAME="Shashi Singh"        # Stores student name
SOFTWARE_CHOICE="Apache HTTP Server" # Stores chosen software

# ----------- System Information Collection -----------------

# Get Linux kernel version
KERNEL=$(uname -r)

# Get current logged-in user
USER_NAME=$(whoami)

# Get system uptime in readable format
UPTIME=$(uptime -p)

# Get current date and time
DATE=$(date)

# Get Linux distribution name from OS release file
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')

# ----------- Display Output ------------------------------

echo "==========================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "==========================================="
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Current User    : $USER_NAME"
echo "Home Directory  : $HOME"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo ""
# Display license information
echo "License Info    : Linux is typically licensed under the GNU General Public License (GPL)."
echo "==========================================="
```

Screenshot of it running:



What you see here is a small program creating a summary of system traits on Linux machines. Grabbing data happens through stored values combined with outputs from terminal tools. Kernel release shows up by running uname -r, while whoami reveals the active account. System runtime appears using uptime -p; current timestamp comes straight from date. Distribution name pulls out lines from /etc/os-release, filtering what matters. Text layout gets shaped via echo, arranging pieces into readable blocks. Learning how scripts handle dynamic inputs becomes clear through these steps. Variables hold snippets, command substitution injects real-time results, formatting keeps it clean.

Script 2 — FOSS Package Inspector (Apache-focused)

Script code:

```
#!/bin/bash
# ==============================================================
# Script 2: FOSS Package Inspector
# Author: Shashi Singh
# Course: Open Source Software
# Description: Checks if a package is installed and displays info
# ==============================================================

# ----------- Variable -------------------------------------
PACKAGE="apache2"   # Package name (Apache HTTP Server)

# ----------- Check Installation ----------------------------
# Check if package is installed using dpkg
if dpkg -l | grep -qw $PACKAGE; then
   echo "$PACKAGE is installed on this system."

   # Display package details (Version, License, Description)
   echo ""
   echo "Package Information:"
   dpkg -l $PACKAGE | grep $PACKAGE
else
   echo "$PACKAGE is NOT installed on this system."
fi
```

```
# ----------- Case Statement (Philosophy Notes) -------------
echo ""
echo "Philosophy Note:"
case $PACKAGE in
   apache2)
      echo "Apache: Backbone of the open web, enabling free and open access to information."
      ;;
   mysql)
      echo "MySQL: Open source database powering millions of applications."
      ;;
   vlc)
      echo "VLC: Media freedom — play anything without restrictions."
      ;;
   firefox)
      echo "Firefox: A browser built to keep the web open and user-focused."
      ;;
   *)
      echo "Unknown package — open source promotes collaboration and freedom."
      ;;
esac
```

Screenshot of it running:

This script checks whether a specified open-source package is installed on the system. It uses an if-else condition along with dpkg -l and grep to verify the presence of the package. If installed, it displays basic package information. A case statement is then used to print a short philosophy note related to the package, demonstrating conditional branching. The script highlights how Linux tools can be used to inspect software and connect technical details with open-source concepts.

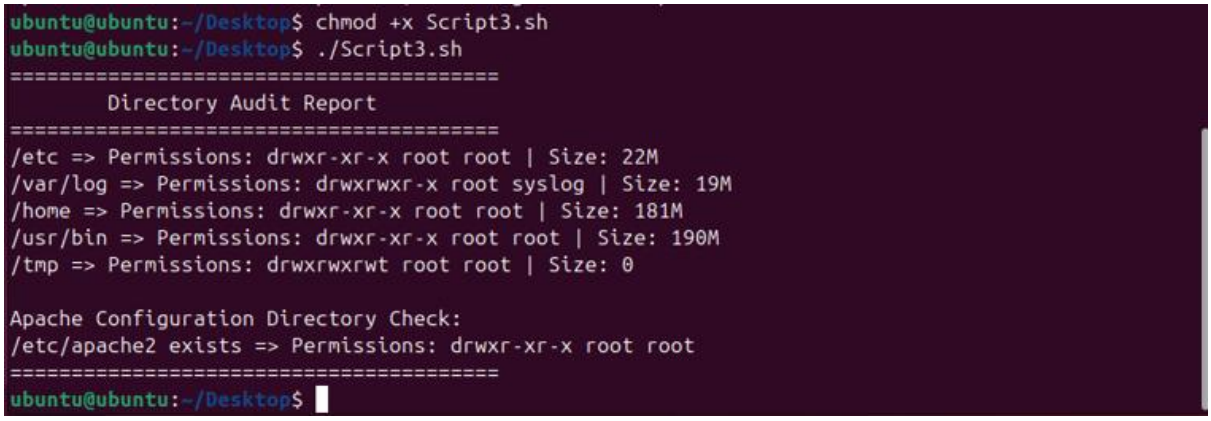
Script 3 — Disk and Permission Auditor

Script code:

```
#!/bin/bash
# ==============================================================
# Script 3: Disk and Permission Auditor
# Author: Shashi Singh
# Course: Open Source Software
# Description: Checks directory size, ownership, and permissions
# ==============================================================
# ----------- Directory List -------------------------------
```

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "        Directory Audit Report"
echo "========================================"

# ----------- Loop Through Directories ---------------------
for DIR in "${DIRS[@]}"; do
   # Check if directory exists
   if [ -d "$DIR" ]; then
      # Get permissions, owner, group
      PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
      # Get size of directory
      SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
      echo "$DIR => Permissions: $PERMS | Size: $SIZE"
   else
      echo "$DIR does not exist on this system"
   fi
done

# ----------- Apache Config Directory Check ----------------
echo ""
echo "Apache Configuration Directory Check:"
CONFIG_DIR="/etc/apache2"

if [ -d "$CONFIG_DIR" ]; then
   PERMS=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
   echo "$CONFIG_DIR exists => Permissions: $PERMS"
else
   echo "$CONFIG_DIR does not exist"
fi
echo "========================================"
```

Screenshot of it running:



This script audits important system directories by checking their disk usage, ownership, and permissions. It uses a for loop to iterate through a predefined list of directories and verifies their existence using an if condition. The ls -ld command is used to extract permissions and ownership details, while du -sh calculates directory size. Additionally, the script checks the Apache configuration

directory to demonstrate application-specific auditing. This script showcases the use of loops, conditional statements, and command-line tools for system analysis.

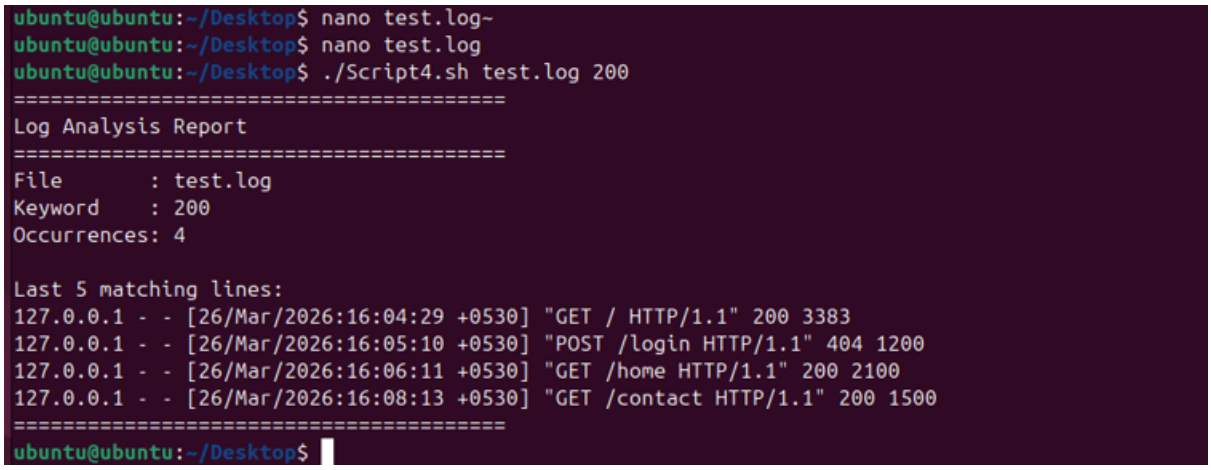
Script 4 — Log File Analyzer

Script Code:

```
#!/bin/bash
# ==============================================================
# Script 4: Log File Analyzer
# Author: Shashi Singh
# Course: Open Source Software
# Description: Counts keyword occurrences in a log file
# ==============================================================
# ----------- Input Arguments -----------------------------
LOGFILE=$1                    # First argument → log file path
KEYWORD=${2:-"error"}         # Second argument → keyword (default: error)
COUNT=0                       # Counter for matches

# ----------- File Check ----------------------------------
# Check if file exists
if [ ! -f "$LOGFILE" ]; then
   echo "Error: File $LOGFILE not found."
   exit 1
fi

# ----------- Empty File Check (Retry Logic) ---------------
if [ ! -s "$LOGFILE" ]; then
   echo "Warning: File is empty. Retrying in 2 seconds..."
   sleep 2
   if [ ! -s "$LOGFILE" ]; then
      echo "File is still empty. Exiting."
      exit 1
   fi
fi

# ----------- Read File Line by Line -----------------------
while IFS= read -r LINE; do
   # Check if line contains keyword (case-insensitive)
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

# ----------- Output Summary -------------------------------
echo "=========================================="
echo "Log Analysis Report"
echo "=========================================="
echo "File       : $LOGFILE"
echo "Keyword    : $KEYWORD"
```

```
echo "Occurrences: $COUNT"
# ----------- Show Last 5 Matching Lines -------------------
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
echo "========================================="
```

Screenshot of it running:



This script analyzes a log file by counting the number of occurrences of a specified keyword. It takes the log file path and keyword as command-line arguments. A while read loop is used to process the file line by line, and an if condition with grep checks for matches in a case-insensitive manner. A counter variable tracks the number of occurrences. The script also includes error handling for missing or empty files and displays the last five matching lines using grep and tail. This demonstrates file handling, loops, conditional logic, and argument passing in shell scripting.

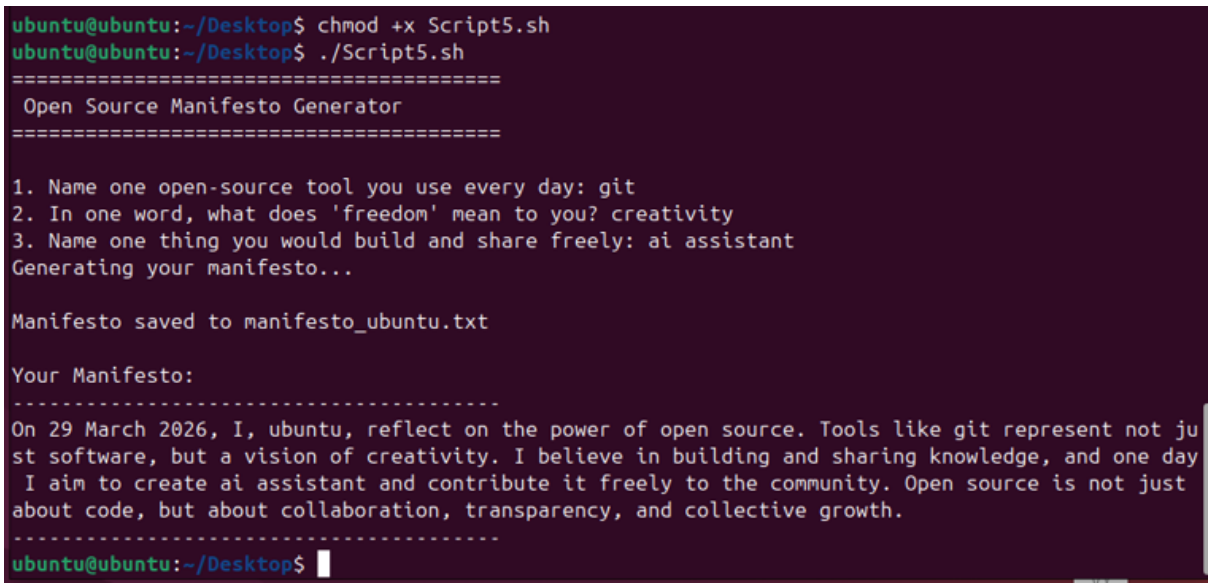
Script 5 — Open Source Manifesto Generator

Script Code:

```
#!/bin/bash
# ================================================================
# Script 5: Open Source Manifesto Generator
# Author: Shashi Singh
# Course: Open Source Software
# Description: Generates a personalized open-source manifesto
# ================================================================
echo "========================================="
echo " Open Source Manifesto Generator"
echo "========================================="
echo ""

# ----------- User Input ----------------------------------
# Take input from user
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
```

```
read -p "3. Name one thing you would build and share freely: " BUILD

# ----------- System Info ----------------------------------
DATE=$(date '+%d %B %Y')          # Current date
USER_NAME=$(whoami)               # Current user
OUTPUT="manifesto_$USER_NAME.txt" # Output file name

# ----------- Generate Manifesto ---------------------------
echo "Generating your manifesto..."
echo ""
echo "On $DATE, I, $USER_NAME, reflect on the power of open source. Tools like $TOOL represent not just software, but a vision of $FREEDOM. I believe in building and sharing knowledge, and one day I aim to create $BUILD and contribute it freely to the community. Open source is not just about code, but about collaboration, transparency, and collective growth." > $OUTPUT

# ----------- Display Output -------------------------------
echo "Manifesto saved to $OUTPUT"
echo ""
echo "Your Manifesto:"
echo "----------------------------------------"
cat $OUTPUT
echo "----------------------------------------"
# ----------- Note on Alias (concept demonstration) --------
# Example alias (not executed):
# alias manifesto="cat $OUTPUT"
```

Screenshot of it running:



This script generates a personalized open-source manifesto by taking user input through the read command. It collects responses about tools, freedom, and ideas, and combines them into a structured paragraph using string concatenation. The output is saved to a text file using output redirection (>). The script also uses system commands like date and whoami to include dynamic information. Additionally, an example of an alias is provided in comments to demonstrate command shortcuts. This script highlights user interaction, file handling, and dynamic content generation in shell scripting.